

PORTADA

Unidad 7 — Desarrollo Web

Introducción al Desarrollo Web y HTML Básico

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



ENCUADRE

El mapa de la Unidad 7

Esta es la primera de **8 sesiones** dedicadas al desarrollo web, la última unidad del curso. El recorrido va del **frontend al backend**: primero construimos la interfaz que ve el usuario, y al final conectamos esa interfaz con un servidor real.

Bloque	Qué se aprende
HTML	Estructura del contenido de una página (hoy y la próxima sesión)
CSS3	Estilos, layout y diseño responsivo
JavaScript	Interactividad y manipulación de la página en el navegador
Spring Boot	Framework de backend en Java para servir datos reales

La unidad cierra con un **Laboratorio Final integrador**, que vale el **20% de la nota** del curso — hoy solo lo mencionamos como el destino del recorrido, sus detalles se explican más adelante.



APERTURA

Cada página que abres hoy —el correo, las redes sociales, la banca en línea— es, en el fondo, HTML, CSS y JavaScript que tu navegador descarga, interpreta y dibuja en milisegundos.



TRANSICIÓN

¿Qué significa realmente "desarrollar para la web"?

Detrás de esa página que se dibuja en milisegundos hay dos mundos de trabajo distintos, que a menudo se confunden cuando alguien empieza en desarrollo web: el **frontend** y el **backend**. Entender esa división es el primer paso antes de escribir una sola etiqueta HTML.

CONCEPTO

Frontend vs. backend

El **desarrollo web** se divide en dos capas de trabajo que colaboran, pero se ejecutan en lugares distintos:

	Frontend	Backend
Dónde corre	En el navegador del usuario	En un servidor
Con qué se construye	HTML, CSS, JavaScript	Un lenguaje de servidor (en esta unidad, Java con Spring Boot)
De qué se encarga	Lo que el usuario ve e interactúa	Lógica de negocio, datos, seguridad

Las próximas dos sesiones son 100% **frontend**: HTML, el lenguaje que le da estructura al contenido de cualquier página.

CONCEPTO

Arquitectura cliente-servidor

Cuando escribes una URL y presionas Enter, tu navegador actúa como **cliente**: envía una petición a una computadora remota, el **servidor**, que responde con el contenido de la página. Es el mismo patrón que pedir comida por teléfono: tú (cliente) haces el pedido, el restaurante (servidor) lo prepara y te lo envía de vuelta.

- **Cliente:** el navegador (Chrome, Firefox, Safari, Edge) que corre en tu computador o celular.
- **Servidor:** una computadora, casi siempre remota, que guarda los archivos de la página y responde peticiones.
- Un mismo servidor puede responder a miles de clientes al mismo tiempo.

CONCEPTO

El protocolo HTTP/HTTPS

Cliente y servidor se hablan siguiendo un protocolo llamado **HTTP** (HyperText Transfer Protocol) — o su versión cifrada, **HTTPS**. Es un protocolo de **petición-respuesta**: el cliente pide algo específico y el servidor contesta.

Verbo HTTP	Para qué se usa
GET	Pedir datos o una página (ej. cargar el inicio de un sitio)
POST	Enviar datos al servidor (ej. enviar un formulario de registro)

El servidor siempre responde con un **código de estado**: `200` significa "todo salió bien", `404` significa "no encontré lo que pediste". Volverás a ver estos códigos en la sesión de JavaScript, cuando el navegador haga estas peticiones por ti.

CONCEPTO

¿Qué hace el navegador con lo que recibe?

El servidor responde con texto plano: código HTML, CSS y JavaScript. El navegador trae un **motor de renderizado** (Blink en Chrome, Gecko en Firefox, WebKit en Safari) que:

1. **Lee** (parsea) el HTML de arriba hacia abajo.
2. **Interpreta** cada etiqueta y decide qué representa: un título, un párrafo, una imagen.
3. **Dibuja** el resultado en pantalla, aplicando después los estilos CSS.

Cómo el navegador organiza internamente esa página (el "DOM") es tema de la sesión de JavaScript, más adelante en esta unidad — hoy nos quedamos con la idea general.

TRANSICIÓN

De la arquitectura general al lenguaje que hoy vamos a escribir

Ya sabes qué es un cliente, qué es un servidor y qué hace el navegador con lo que recibe. Ahora toca escribir el primer ingrediente de esa respuesta: **HTML**, el lenguaje que estructura el contenido de toda página web.

CONCEPTO

¿Qué es HTML?

HTML (HyperText Markup Language) es un **lenguaje de marcado**, no un lenguaje de programación: no tiene variables, condicionales ni ciclos. Su trabajo es **describir la estructura** del contenido usando **etiquetas** (`<tag>`) que pueden llevar **atributos** (pares nombre-valor dentro de la etiqueta).

HTML existe desde 1991, y su versión actual, **HTML5**, es el estándar vigente desde 2014, mantenido por el W3C y el WHATWG. Es compatible con todos los navegadores modernos.



CONCEPTO

Estructura mínima de un documento HTML

Todo archivo HTML válido sigue el mismo esqueleto: una declaración de tipo de documento, una etiqueta raíz `<html>` , una sección `<head>` con metadatos que no se ven en pantalla, y una sección `<body>` con el contenido visible.

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mi primera página</title>
</head>
<body>
  <h1>Hola, mundo</h1>
</body>
</html>
```

`<!doctype html>` le dice al navegador que use el estándar HTML5. `<title>` define lo que aparece en la pestaña del navegador.

CONCEPTO

Etiquetas de texto básicas

Etiqueta	Para qué sirve
<h1> ... <h6>	Títulos y subtítulos, de mayor (h1) a menor (h6) jerarquía
<p>	Un párrafo de texto
	Texto con importancia (se ve en negrita)
	Texto con énfasis (se ve en cursiva)
 	Salto de línea dentro de un mismo bloque

Los **comentarios** se escriben con `<!-- así -->` : el navegador los ignora al renderizar, pero son útiles para dejar notas en el código.

CONCEPTO

Listas: ordenadas y no ordenadas

HTML tiene dos tipos de lista, ambas construidas con elementos `<li>` (list item) dentro de un contenedor:

```
<ul>
  <li>Sin orden específico</li>
  <li>Se muestra con viñetas</li>
</ul>

<ol>
  <li>Primer paso</li>
  <li>Segundo paso</li>
</ol>
```

`<ul>` (unordered list) se usa cuando el orden no importa; `<ol>` (ordered list) cuando sí, porque el navegador la numera automáticamente.

CONCEPTO

Enlaces e imágenes

Dos de las etiquetas más usadas de HTML conectan una página con el resto de la web:

```
<a href="https://ejemplo.com" target="_blank">
  Visitar sitio
</a>


```

- `<a href="...">` crea un enlace; el atributo `target="_blank"` lo abre en una pestaña nueva.
- `` inserta una imagen; **no tiene etiqueta de cierre**.

CONTRAEJEMPLO

El atributo `alt` no es opcional

```
<!-- Mal: sin alt, inaccesible -->


<!-- Bien: alt describe el contenido -->

```

Sin `alt`, una persona que usa un **lector de pantalla** no tiene forma de saber qué muestra la imagen, y si la imagen no carga, el navegador no tiene qué texto mostrar en su lugar. Por eso `alt` es **obligatorio por accesibilidad**, no un extra — es un concepto al que volvemos con más detalle en la próxima sesión.

CONCEPTO

Atributos globales y buenas prácticas

Además de `href` , `src` o `alt` (específicos de cada etiqueta), existen **atributos globales** que cualquier etiqueta puede llevar, como `id` , `class` o `lang` — los verás en detalle en la sesión de CSS, cuando sirvan para aplicar estilos.

- **Anida correctamente:** una etiqueta que abre dentro de otra debe cerrar antes que ella.
- **Cierra siempre las etiquetas** que lo requieren (`<p>...</p>`), aunque el navegador sea tolerante con errores.
- **Indenta el código:** cada nivel de anidamiento, dos espacios más — hace el archivo legible.
- **Nombres de archivo en minúsculas**, sin espacios ni tildes (ej. `index.html` , no `Índice.html`).

INTERACCIÓN

Escriban su primera página

En parejas, en papel o en un editor de texto: escriban a mano la estructura completa de una página HTML simple que tenga:

- Un título principal con `<h1>` .
- Un párrafo con `<p>` que se presente.
- Una lista (`<ul>` u `<ol>`) con al menos 3 elementos.
- Una imagen con `<img>` , incluyendo su atributo `alt` .

8 minutos · no olviden la estructura mínima completa (doctype, html, head, body)

SÍNTESIS

Lo que hay que llevarse de hoy

1. El desarrollo web se divide en **frontend** (navegador) y **backend** (servidor).
2. La arquitectura **cliente-servidor** funciona por petición-respuesta, usando el protocolo **HTTP/HTTPS**.
3. El navegador **parsea** el HTML recibido y lo dibuja en pantalla.
4. **HTML** es un lenguaje de marcado: etiquetas y atributos, no lógica de programación.
5. Todo documento HTML sigue la misma estructura mínima: `doctype` , `html` , `head` , `body` .
6. El atributo `alt` en `<img>` es obligatorio por accesibilidad.



CIERRE

Antes de la próxima sesión

- Practicar la estructura mínima de HTML escribiendo una página propia con las etiquetas vistas hoy.
- Guardarla con un nombre de archivo en minúsculas, sin tildes ni espacios.

Próxima sesión: HTML — Formularios, Tablas y Semántica — profundizamos en cómo capturar datos del usuario, organizar información tabular y usar etiquetas semánticas para accesibilidad.

